ChatGPT

GEDCOM Normalization and Enrichment System Design

Personal Name Parsing and Normalization

The system must **split and tag each NAME field** into its components. For example, the GEDCOM standard shows how "Lt. Cmndr. Joseph /Allen/ jr." is parsed into NPFX="Lt. Cmndr.", GIVN="Joseph", SURN="Allen", and NSFX="jr" [1]. Similarly, "SGT Mr. Norman Forsyth Menzies Jr" should be parsed so that military rank and honorifics become prefix/title tags, "Norman" becomes GIVN (given name) and "Menzies" becomes SURN (surname), with "Forsyth" handled as an additional given/middle name and "Jr" as NSFX (suffix). A dictionary or dataset of known prefixes/suffixes (e.g. military ranks, honorifics, nicknames) should be used to recognize and classify tokens (e.g. mapping "SGT" to a rank or occupation). If a token does not fit a standard GEDCOM field (e.g. unusual title or rank), it may be stored in a custom tag or an individual attribute structure (GEDCOM's FACT).

Splitting names is inherently tricky — some surnames or prefixes are multi-word, and cultural ordering varies. As one analysis notes, "splitting names is harder than it looks" and recommends capturing first/last names in separate fields to avoid ambiguity [2]. In practice, the parser should apply heuristics (e.g. known prefix lists, capitalization rules, linguistic rules) and allow manual override. All parsed parts (NPFX, GIVN, NICK, SPFX, SURN, NSFX) should be output even if originally absent, enriching incomplete input files. For instance, if a GEDCOM name has no child tags, the processor will still write out GIVN/SURN/NSFX etc. based on the split. In summary, the NAME string is the "primary" name and name pieces are optional auxiliaries [1]; this system will generate and normalize both for consistency.

Place and Geolocation Standardization

Places should be **standardized and geocoded**. GEDCOM's PLAC tag value (a comma-separated location string) should be parsed into structured components (city, county, state, country). When latitude/longitude are present under MAP/LATI/LONG for an event, those coordinates should be stored. (GEDCOM allows event-specific coordinates but not place-level coordinates [3].) To handle events without coordinates, a separate *gazetteer* of places can be maintained: e.g. a database of towns with canonical names and lat/long. As the webtrees FAQ explains, storing coordinates in a shared gazetteer avoids misattributing an event's exact location to a generic place center [4].

This system should use authoritative geo-datasets for enrichment: for example, the GeoNames database covers all countries with 11+ million place names [5], and FamilySearch's Places API offers a database of over 6 million standardized locations [6]. During import, each PLAC can be looked up (by name/jurisdiction) against these sources or a geocoding API, and standardized names (and geo-coordinates) applied. In PostgreSQL, a **PostGIS** extension can store location points and support spatial queries (e.g. finding nearest place, mapping) [7]. Every GEDCOM event or place record in the database should reference the canonical place record (with lat/long and possibly a unique identifier), ensuring consistency (e.g. "Paris, Texas, USA" always resolves to the same record).

Occupation and Additional Attributes Extraction

Occupations and other attributes should be **extracted into their proper tags**. If a GEDCOM file has an `OCCU` tag on an individual, use it directly ("occupation – the type of work or profession of an individual" [8]). If not, the system should scan notes or other text fields for keywords (e.g. a note line "Occupation: Farmer") and parse out the value. A regex or simple NLP pass can detect patterns like `Occupation: <job>` and populate OCCU. Similarly, other individual attributes (CAST for caste, RELI for religion, etc.) should be filled from any available structured or semi-structured text. The system can also employ a generic attribute structure (GEDCOM's FACT with a TYPE) for any data not covered by standard tags. All tags and extracted values are parameterized (optional), so users can enable or disable enrichment rules (e.g. skip extracting from notes if undesired).

External Data Enrichment (APIs and Datasets)

The pipeline will integrate multiple **external data sources and APIs** to enrich the data. In addition to the author's own normalization datasets (military ranks, name prefixes, occupation lists, etc.), the system should optionally query public resources. For example, FamilySearch offers free APIs for family trees, historical records, and especially its Places database [6]. Using the FamilySearch Places API can improve place name matching. Similarly, Wikidata (a CC0 knowledge base) can be used to link persons or places to unique identifiers: its API allows browsing and querying of entities [9]. (By matching an ancestor to a Wikidata QID, one can inherit structured info like alternate names, birth/death dates, etc.)

For geocoding, services like GeoNames or OpenStreetMap/Nominatim can be used. The system will include a command-line client or modules for these lookups, with API keys configurable. All enrichment steps should be optional and cacheable (to avoid overloading services). The design will use an **OpenAPI/REST** interface for web integrations (as suggested by the `openapi` folder). In summary, the tool will enrich data via all available means: local datasets, genealogical APIs (FamilySearch, etc.), linked-data (Wikidata), and GIS services, in a modular, parameter-driven way.

Database Design (PostgreSQL, Neo4j, etc.)

Persist the enriched genealogy data in a robust database architecture. A **relational database (PostgreSQL)** will store core entities (Individuals, Families, Events, Sources, Places, etc.), with normalized tables for attributes (names, occupations, addresses). Geospatial data (coordinates) will use PostGIS types [7]. The relational model is ideal for data validation and bulk queries (e.g. SQL reports).

In parallel, a **graph database (Neo4j)** will model the family tree: each individual is a node, with `PARENT_OF`, `SPOUSE_OF`, etc. relationships. As Neo4j documentation notes, "family trees are much better handled as a graph than as a bunch of text fragments" [10]. The graph allows easy traversal of relationships (e.g. ancestors, descendants, complex kinship), and can be populated from the same normalized data. Alternatively, both PostgreSQL and Neo4j could be used side-by-side, each optimized for different query patterns.

Data consistency will be enforced by the schema: required fields, unique constraints (e.g. GEDCOM IDs), and referential integrity for cross-references. The database design should follow best practices (3NF for

relational tables, meaningful indexes on identifiers and foreign keys). All fields, including empty child tags, should be explicitly stored or have nulls, to ensure full normalization as required.

Command-Line Interface, APIs and Validation

A **command-line (CLI) toolset** will provide import, normalization, and export capabilities. For example: `gedcom import`, `gedcom normalize`, `gedcom export`, and `gedcom query`. The query mode might accept simple expressions (as in the Neo4j example), enabling users to extract CSV or JSON of selected facts. The CLI should accept parameters (via YAML/JSON config files) to enable/disable steps like name parsing, geocoding, occupation extraction, etc.

Imported GEDCOM files will first be **validated**. The tool should check for GEDCOM syntax errors and missing required tags (e.g. validate against the 5.5.5 spec). Existing GEDCOM validators (like GEDInline or Chronoplex Validator [11]) could be referenced or integrated. The validation must ensure cross-references (INDI<->FAM) are consistent. Any normalization or enrichment step should include sanity checks (e.g. if place lookup fails, log a warning).

Automated **testing** is crucial. The codebase should include unit tests (in `tests/data`) covering diverse cases: multi-part names, unusual titles, foreign placenames, etc. Integration tests should import sample GEDCOMs, run the full pipeline, and verify outputs (e.g. all expected child tags appear, known place IDs are assigned). Continuous integration tools should enforce code quality and spec adherence.

User Interface and Audience Considerations

Since the end-users are genealogy enthusiasts and researchers, the interface must be **domain-aware**. The eventual GUI (web-based) should present data in genealogical terms (pedigree charts, family tables, maps of ancestral migrations, etc.). Inputs and outputs will use familiar GEDCOM terminology (e.g. “Given Name” field instead of raw GIVN). The system should not hide technical details from expert users, but should allow novices to run default normalization. Clear documentation and help (tooltips, manuals) must explain any custom behavior (e.g. how “SGT” was classified). Logging and traceability are important: for each enriched field, note the source (e.g. “Occupation was parsed from NOTE ‘Occupation: Farmer’”).

Configuration, Templates, and Documentation

The project will use **YAML/JSON configuration templates** to make the processing pipeline context-aware and flexible. For example, GEDCOM-to-database mapping, normalization rules, and API credentials can be stored in YAML files. OpenAPI or similar schemas (in `openapi/`) will define any REST endpoints (for web UI or third-party use). In-code documentation and a living design document will capture this context, enabling automated tools (or the CLI) to resume processing with full knowledge of settings.

Standards, Best Practices, and Specifications

All processing will adhere to genealogical data standards. The GEDCOM 5.5.5 specification is the baseline: name pieces (NPFX, GIVN, SURN, NSFX) are strictly defined [1] [8], as are event tags (e.g. BIRT, MARR) and attribute tags (OCCU, TITL). The system should conform to these standards, using their definitions of each

tag. Where the spec lacks a field (e.g. certain cultural name elements), use well-documented extensions or GEDCOM X alternatives. For place names and dates, follow international standards (e.g. ISO country codes, UTC dates).

Best practices from software engineering will apply: version control, code reviews, modular design, and performance tuning (e.g. indexing spatial data for quick place lookups). Sensitive data (living persons) should be handled per privacy standards (e.g. exclude living people's details in outputs by default).

All design decisions, data models, and workflows will be documented in detail. The documentation will include data flow diagrams, schema diagrams, and rationale for choices (for example, why "SGT" was treated as a rank vs. a title). By the planned launch, the system's architecture – from ingestion to enrichment to storage and interface – will be fully specified, following the goals of complete GEDCOM standardization and enrichment laid out here【21†】 (even if we cannot cite that directly, it matches our objectives).

Sources: The design principles above draw on the GEDCOM specification and real-world examples. For instance, the GEDCOM spec explicitly shows name-piece tags (NPFX, GIVN, SURN, NSFX) in use [1]. Studies of genealogical data note the challenges of name parsing [2] and the value of treating family trees as graph data [10]. Geodata and enrichment sources (GeoNames [5], FamilySearch Places [6], PostGIS [7]) provide the foundation for our plan. By combining these sources and best practices, the system will robustly parse, normalize, enrich, and serve GEDCOM data.

---

[1] [8] The GEDCOM Standard
https://gedcom.io/specifications/gedcom7-rc.pdf

[2] python - Splitting a person's name into forename and surname - Stack Overflow
https://stackoverflow.com/questions/259634/splitting-a-persons-name-into-forename-and-surname

[3] [4] FAQ - How do locations work?
https://webtrees.net/faq/locations/

[5] GeoNames
https://www.geonames.org/

[6] Level Up Your Genealogy Apps and Services | FamilySearch API
https://developers.familysearch.org/

[7] PostGIS: A powerful geospatial extension for PostgreSQL | Red Hat Developer
https://developers.redhat.com/articles/2025/10/02/postgis-powerful-geospatial-extension-postgresql

[9] Wikidata
https://research.google.com/pubs/archive/42240.pdf

[10] Discover AuraDB Free: Importing GEDCOM & Genealogy Data
https://neo4j.com/blog/developer/discover-auradb-free-importing-gedcom-files-and-exploring-genealogy-ancestry-data-as-a-graph/

[11] GEDCOM Validation - Tamura Jones
https://www.tamurajones.net/GEDCOMValidation.xhtml